Reviewer Pack — Noise-Stability Plateau at rho=1/3 Bounds DPLL Leaves on 3-C...

Entry 3385dd792f63 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 09:18:06 UTC

Noise-Stability Plateau at rho=1/3 Bounds DPLL Leaves on 3-CNF

Entry ID: `3385dd792f63` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 09:18:06 UTC

1. Conjecture

Field A (mathematical branch): Fourier analysis of Boolean functions (noise stability Stab_rho and Bonami-Beckner noise operator T_rho) **Field B** (complexity object): DPLL search-tree leaf count on random 3-CNF formulas

Statement:

> For a 3-CNF formula F on n variables with indicator function f_F:{-1,1}^n -> {0,1} (1 iff assignment satisfies F), let S(F) := Stab_{1/3}(f_F) - (E[f_F])^2 = sum_{|T|>=1} (1/3)^{|T|} hatf(T)^2. We conjecture that for every unsatisfiable 3-CNF F with m clauses on n<=20 variables, the number L(F) of leaves explored by the standard unit-propagating DPLL with fixed lexicographic branching satisfies L(F) >= 2^{n} * S(F) / (3 * m), and equivalently for satisfiable F we get L(F) <= 2^{n} * (1 - 9*S(F)/m). One satisfiability instance violating either inequality refutes the conjecture.

Rationale (proposer's reasoning):

> At noise rate 1/3 each clause's local indicator (a degree-3 Boolean function on three variables) decays by exactly (1/3)^3=1/27 per level, matching the branching factor of DPLL on a 3-CNF; thus the rho=1/3 noise stability isolates the 'clause-coupled' Fourier mass that DPLL must shatter. Kahn-Kalai-Linial-style influence bounds show that each clause contributes at most O(1/n) to total influence, so summing Stab_{1/3} over clauses should track tree size up to the clause-density factor m. This bridges hypercontractive Fourier weight with concrete branching cost, a connection rarely tested empirically.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4e6cd17a868e6b4f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all generated 3-CNF instances (7 n-values × 3 densities × 50 seeds = 1050 instances), each instance must satisfy its respective inequality (UNSAT: L(F) >= 2^n*S(F)/(3m); SAT: L(F) <= 2^n*(1-9*S(F)/m)) within 3-sigma Monte-Carlo error on S(F).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - noise stability Boolean function DPLL 3-CNF complexity - Fourier analysis satisfiability random 3-SAT search tree lower bound - Bonami-Beckner noise operator k-CNF DPLL leaves branching

Top relevant hits considered: - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/1112.2313v2] QBF-Based Boolean Function Bi-Decomposition - [http://arxiv.org/abs/1402.4455v1] Symbiosis of Search and Heuristics for Random 3-SAT - [http://arxiv.org/abs/0710.0805v3] On the Satisfiability Threshold and Clustering of Solutions of Random 3-SAT Formulas - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/math/9906169v1] Regular Operators on Hilbert C^*-modules - [http://arxiv.org/abs/1510.0 Two types of branching programs with bounded repetition that cannot efficiently compute monotone 3-CNFs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from collections import defaultdict

def generate_3cnf(n, m):
    clauses = []
    for _ in range(m):
        clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(3)]
        clauses.append(clause)
    return clauses

def evaluate_formula(f_F, assignment):
    return all(any(f_F[var] == literal for var, literal in zip(clause, assignment)) for clause in

def compute_stab_1_3(f_F):
```

```
    n = len(next(iter(f_F)))
    rho = 1 / 3
    total = 0
    samples = 4000
    for _ in range(samples):
        x = [random.choice([-1, 1]) for _ in range(n)]
        y = [x[i] if random.random() < rho else -x[i] for i in range(n)]
        total += f_F[tuple(x)] * f_F[tuple(y)]
    return total / samples

def dpll(f_F, assignment, clauses):
    if not clauses:
        return 1
    var = next(iter(clauses[0]))
    for literal in [-1, 1]:
        new_assignment = assignment + [literal]
        new_clauses = [c for c in clauses if literal * c[var-1] != -1]
        if all(literal * f_F[tuple(a)] != -1 for a in new_assignment):
            result = dpll(f_F, new_assignment, new_clauses)
            if result > 0:
                return result
    return 0

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16, 18, 20]
    densities = [3.0, 4.26, 5.0]
    results = []

    for n in n_values:
        for alpha in densities:
            m = int(alpha * (2 ** n))
            for _ in range(50):
                clauses = generate_3cnf(n, m)
                f_F = {tuple([random.choice([-1, 1]) for _ in range(n)]): evaluate_formula(f_F, as
                S_F = compute_stab_1_3(f_F)
                L_F = dpll(f_F, [], clauses)

                if not f_F.values():
                    continue

                expected_bound = (2 ** n) * S_F / (3 * m)
                if all(f_F.values()):
                    bound = (2 ** n) * (1 - 9 * S_F / m)
                else:
                    bound = (2 ** n) * S_F / (3 * m)

                if f_F.values():
                    if L_F < expected_bound:
                        results.append({"n": n, "alpha": alpha, "L_F": L_F, "S_F": S_F, "bound": b
                else:
                    if L_F > bound:
                        results.append({"n": n, "alpha": alpha, "L_F": L_F, "S_F": S_F, "bound": b

    metric_value = sum(result["L_F"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
```

```
    return {
        "metric_name": "L(F)",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction == 1.0,
        "counterexample": "" if support_fraction == 1.0 else results[0]["counterexample"]
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    results = [run_trial(seed) for seed in seeds]
    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction == 1.0:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_689eea86.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_689eea86.py", line 62, in run_trial
    f_F = {tuple([random.choice([-1, 1]) for _ in range(n)]): evaluate_formula(f_F, assignment) for ass
1, 1], repeat=n)}
                                                                                    ^^^
UnboundLocalError: cannot access local variable 'f_F' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with an UnboundLocalError before producing any data, so no instances were evaluated and neither the support nor falsification condition can be assessed. | next: Fix the test harness bug (initialize f_F properly before constructing the assignment-to-evaluation dictionary) and rerun the full 1050-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	18625
2	preregistration	claude_max	opus	0	0	6855
3	novelty	claude_max	opus	0	0	3052
4	novelty	claude_max	opus	0	0	9137
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19198
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13054
7	judge	claude_max	opus	0	0	5582

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 75503 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/3385dd792f63.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3385dd792f63.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3385dd792f63.tar.gz` (if generated)